Data Science Fundamentals Assessment

Data Engineer Internship, Task 1

Name: Santosh | Notebook: Task1_DS_Fundamentals.ipynb

1. Introduction

This report covers the fundamentals of Data Science that I practiced using Python in Google Colab. It includes Python programming basics, statistics, data types and structures, and clean coding practices. All code is in the notebook Task1_DS_Fundamentals.ipynb, and every output shown below comes from my own run of that notebook.

2. Data Science Lifecycle

Data Science work follows a set of stages:

- Define the problem and what a good answer looks like.
- Collect the data from reliable sources.
- Clean the data (missing values, duplicates, wrong types).
- Explore and analyze it to find patterns.
- Build a model or draw conclusions.
- Communicate the results with charts and recommendations.

It is used in many industries, for example fraud detection in finance, demand forecasting in retail, patient trend analysis in healthcare, and usage prediction in the energy sector.

3. Python Programming Basics

- Variables of type string, integer, float and boolean, checked with type().
- Arithmetic operators: +, -, *, /, //, %, **.
- if / elif / else to assign a grade from marks (72 marks gave Pass).
- for and while loops, and a reusable average() function.
- Lists, tuples, dictionaries and sets, plus file handling (write then read a text file).

3.1 Variables, operators, conditions, loops, functions

Code

```
 # --- Variables, data types, operators ---
name = "Santosh"          # string
age = 19                  # integer
cgpa = 7.5                # float
is_student = True         # boolean
print(name, age, cgpa, is_student)
print(type(name), type(age), type(cgpa), type(is_student))

# operators
a, b = 10, 3print(a + b, a - b, a * b, a / b, a // b, a % b, a ** b)

# --- Conditional statements ---
marks = 72
```

```
if marks >= 75:
    grade = "Distinction"
elif marks >= 40:
    grade = "Pass"
else:
    grade = "Fail"
print("Grade:", grade)

# --- Loops ---
for i in range(1, 6):
    print("Square of", i, "is", i * i)

count = 0
while count < 3:
    print("while loop count:", count)
    count += 1

# --- Functions ---
def average(numbers):
    """Return the average of a list of numbers."""
    return sum(numbers) / len(numbers)

print("Average:", average([10, 20, 30, 40]))
```

Output

```
Santosh 19 7.5 True
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
13 7 30 3.3333333333333335 3 1 1000
Grade: Pass
Square of 1 is 1
Square of 2 is 4
Square of 3 is 9
Square of 4 is 16
Square of 5 is 25
while loop count: 0
while loop count: 1
while loop count: 2
Average: 25.0
```

3.2 Lists, tuples, dictionaries, sets and file handling

Code

```
# list (changeable)
subjects = ["Maths", "Python", "DBMS"]
subjects.append("OS")
print(subjects)

# tuple (not changeable)
point = (10, 20)
```

```
print(point[0], point[1])

# dictionary (key: value)
student = {"name": "Santosh", "branch": "CMPN", "year": 2}
student["cgpa"] = 7.5
print(student)
for key, value in student.items():
    print(key, "->", value)

# set (unique values only)
nums = {1, 2, 2, 3, 3, 3}
print(nums)

# file handling: write, then read
with open("notes.txt", "w") as f:
    f.write("Data Science is fun\nLine two")

with open("notes.txt", "r") as f:
    print(f.read())
```

Output

```
['Maths', 'Python', 'DBMS', 'OS']
10 20
{'name': 'Santosh', 'branch': 'CMPN', 'year': 2, 'cgpa': 7.5}
name -> Santosh
branch -> CMPN
year -> 2
cgpa -> 7.5
{1, 2, 3}
Data Science is fun
Line two
```

A list can be changed, a tuple cannot, a dictionary stores key and value pairs, and a set keeps only unique values (the set {1, 2, 2, 3, 3, 3} became {1, 2, 3}).

4. Statistical Foundations

I used a sample of 15 student marks and calculated the descriptive statistics below.

Code

```
import numpy as np
import pandas as pd

# sample data: marks of 15 students
marks = [45, 56, 67, 72, 72, 78, 80, 82, 85, 88, 90, 90, 90, 95, 98]
s = pd.Series(marks)

print("Mean:", s.mean())
print("Median:", s.median())
```

```
print("Mode:", s.mode().tolist())
print("Variance:", round(s.var(), 2))
print("Std Dev:", round(s.std(), 2))
print("25th percentile:", s.quantile(0.25))
print("50th percentile:", s.quantile(0.50))
print("75th percentile:", s.quantile(0.75))

# --- Probability basics ---
# chance that a random student scored 80 or more
p = (s >= 80).sum() / len(s)
print("P(marks >= 80):", round(p, 2))

# --- Correlation ---
study_hours = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
corr = np.corrcoef(study_hours, marks)[0, 1]
print("Correlation (hours vs marks):", round(corr, 2))
```

Output

```
Mean: 79.2
Median: 82.0
Mode: [90]
Variance: 216.74
Std Dev: 14.72
25th percentile: 72.0
50th percentile: 82.0
75th percentile: 90.0
P(marks >= 80): 0.6
Correlation (hours vs marks): 0.94
```

Interpretation

- Mean = 79.2 and median = 82.0. The mean is lower than the median, which shows that a few low marks (45, 56) pull the average down.
- Mode = 90, the most repeated mark.
- Standard deviation = 14.72, so marks are fairly spread around the mean.
- Percentiles: 25th = 72, 50th = 82, 75th = 90. Half the students scored 82 or more.
- Probability: P(marks >= 80) = 9/15 = 0.6.
- Correlation between study hours and marks = 0.94, a strong positive relationship in this sample data.

5. Data Types and Structures

I built a DataFrame holding five kinds of data and converted each column to the correct type, so that the right operations work on it.

Column	Data type	Example values	Pandas handling
student_id	Numerical	101, 102, 103	int64
branch	Categorical	CMPN, IT, EXTC	astype('category')

performance	Ordinal	Poor < Average < Good < Excellent	ordered Categorical
join_date	Datetime	2024-07-01	pd.to_datetime()
feedback	Text	great course, too fast	object, .str methods

Code

```
df = pd.DataFrame({
    "student_id": [101, 102, 103, 104, 105],                # numerical
    "branch": ["CMPN", "IT", "CMPN", "EXTC", "IT"],         # categorical
    "performance": ["Good", "Average", "Excellent", "Poor", "Good"],  # ordinal
    "join_date": ["2024-07-01", "2024-07-02", "2024-07-01", "2024-07-03", "2024-07-02"],  # datetime
    "feedback": ["great course", "too fast", "loved it", "needs help", "nice labs"]       # text
})

# convert to proper types
df["branch"] = df["branch"].astype("category")
df["performance"] = pd.Categorical(
    df["performance"],
    categories=["Poor", "Average", "Good", "Excellent"],
    ordered=True
)
df["join_date"] = pd.to_datetime(df["join_date"])

print(df.dtypes)
print(df)

# using each type
print(df["join_date"].dt.day_name())             # datetime
print(df["feedback"].str.len())                   # text
print(df[df["performance"] >= "Good"])            # ordinal comparison
print(df["branch"].value_counts())                # categorical
```

Output

```
student_id              int64
```

```
branch                  category
performance             category
join_date         datetime64[ns]
feedback                  object
dtype: object
   student_id branch performance  join_date       feedback
0         101   CMPN        Good 2024-07-01  great course
1         102     IT     Average 2024-07-02       too fast
2         103   CMPN   Excellent 2024-07-01       loved it
3         104   EXTC        Poor 2024-07-03    needs help
4         105     IT        Good 2024-07-02     nice labs
0       Monday
1      Tuesday
2       Monday
3    Wednesday
4      Tuesday
Name: join_date, dtype: object
0    12
1     8
2     8
3    10
4     9
Name: feedback, dtype: int64
   student_id branch performance  join_date       feedback
0         101   CMPN        Good 2024-07-01  great course
2         103   CMPN   Excellent 2024-07-01       loved it
4         105     IT        Good 2024-07-02     nice labs
branch
CMPN    2
IT      2
EXTC    1
Name: count, dtype: int64
```

Because performance is an ordered category, the comparison df['performance'] >= 'Good' correctly returned Good and Excellent rows. Because join_date is a real datetime, I could get the weekday name directly.

6. Data Science Best Practices

- Meaningful variable and function names, with docstrings.
- np.random.seed(42) so the results are reproducible.
- Removed duplicate rows and filled the missing value with the median.
- Reused code through functions instead of repeating it.
- Noted that correlation does not mean causation.

Code

```
import numpy as np
import pandas as pd

np.random.seed(42)    # same results every run (reproducible)

def summarize(column):
    """Print basic stats for a pandas Series."""
    print("Count :", column.count())
```

```
    print("Mean  :", round(column.mean(), 2))
    print("Std   :", round(column.std(), 2))
    print("Min   :", column.min())
    print("Max   :", column.max())

# data with a missing value and a duplicate
data = pd.DataFrame({
    "student_marks": [55, 67, np.nan, 82, 82, 91],
    "attendance_pct": [70, 80, 75, 90, 90, 95]
})

data = data.drop_duplicates()
data["student_marks"] =
data["student_marks"].fillna(data["student_marks"].median())

summarize(data["student_marks"])

# responsible interpretation
corr = data["student_marks"].corr(data["attendance_pct"])
print("Correlation:", round(corr, 2))
print("Note: correlation does not mean causation.")
```

Output

```
Count : 5
Mean  : 73.9
Std   : 13.81
Min   : 55.0
Max   : 91.0
Correlation: 0.91
Note: correlation does not mean causation.
```

After removing the duplicate row, 5 records remained. The missing mark was replaced with the median, which is safer than the mean when data can contain extreme values.

7. Conclusion

This task gave me a solid base in Python, statistics and data handling. I now understand how to choose the right data type for each column, how to summarize data with descriptive statistics, and how to write clean, reproducible code. These skills are the starting point for the data cleaning, analysis and visualization work in the next tasks.

8. GitHub Link

https://github.com/Santoshgutte1522/data-engineer-internship.git